

| 2026-04-15

| Современные архитектуры обработки последовательностей и механизм внимания

До сих пор рассматривались данные, порядок которых не влиял на то, как мы можем их обработать — изображения (пиксели), табличные данные, какие-либо числовые массивы технически независимы друг от друга.

Но в случаях с **последовательностями (sequences)** порядок становится одним из **ключевых** признаков. Две основные категории последовательностей, с которыми работает машинное обучение, — текст и временные ряды.

≡ Example

Кот съел мыш , Мышь съела кота — используемые лексемы совпадают, но смысл разный

..., 23.1, 23.4, 24.0, 24.8, 25.1, 25.3, 24.9, [?], [?], ... — элемент датасета ETTh1 с почасовой температурой масла в трансформаторе; значение температуры в момент t зависит от предыдущих, т.е. последовательность является временным рядом.

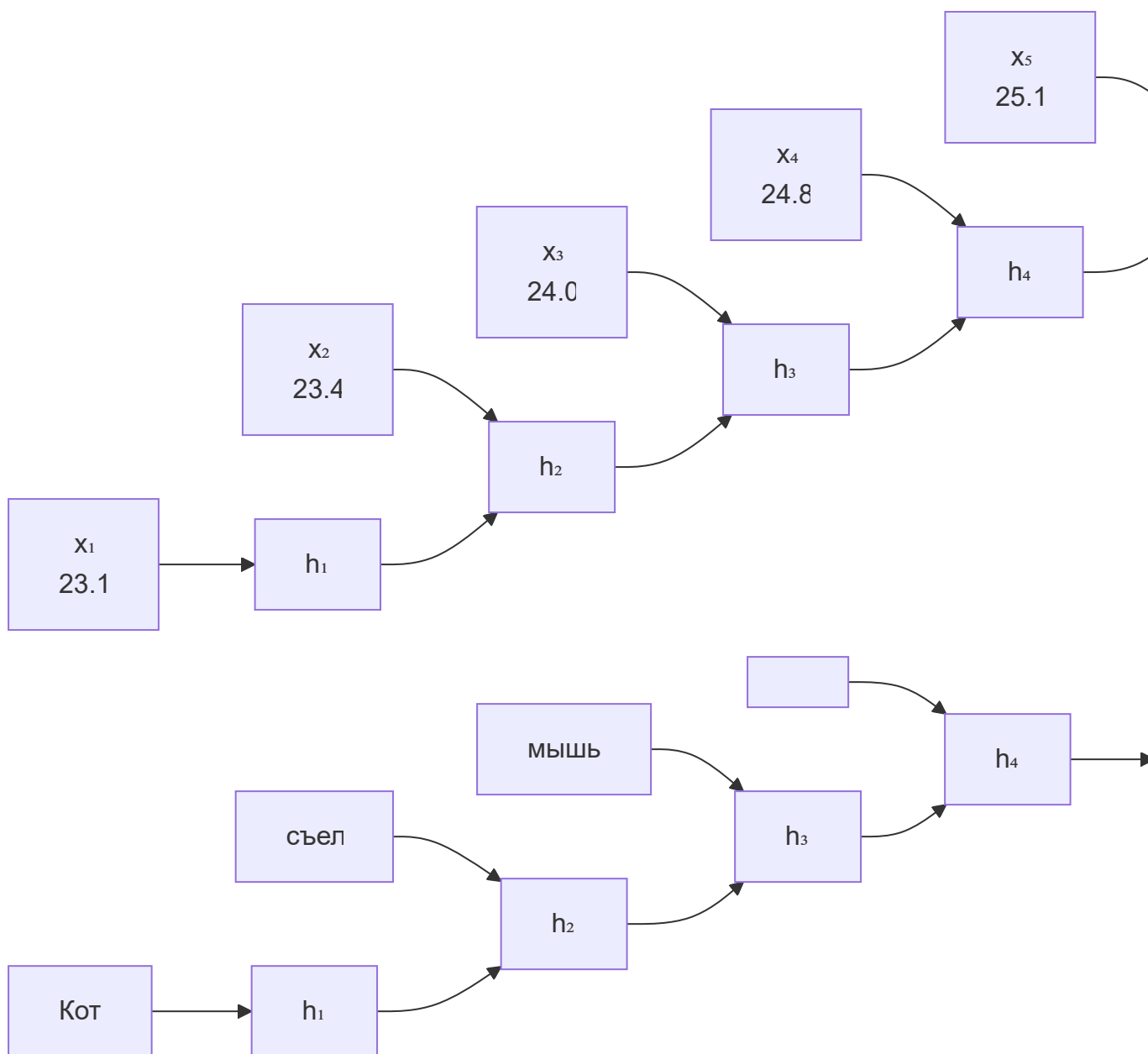
| Рекуррентные сети (RNN, LSTM) и их ограничения

RNN (Recurrent Neural Network) обрабатывает последовательность **шаг за шагом**.

На каждом шаге t :

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

Скрытое состояние h_t — «память» сети. Прогноз строится из h_T — **последнего** состояния.



На каждом шаге t сеть обновляет свое скрытое состояние (hidden state) h_t , используя текущее слово/вход x_t и скрытое состояние с предыдущего шага h_{t-1} .

Tip

Продвинутые варианты RNN: **LSTM** (Long Short-Term Memory) и **GRU** (Gated Recurrent Unit).

LSTM решает проблему затухания градиентов, добавляя **ячейку памяти** c_t с вентилями:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{forget gate})$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{input gate})$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_c[h_{t-1}, x_t] + b_c)$$

$$h_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \odot \tanh(c_t) \quad (\text{output gate})$$

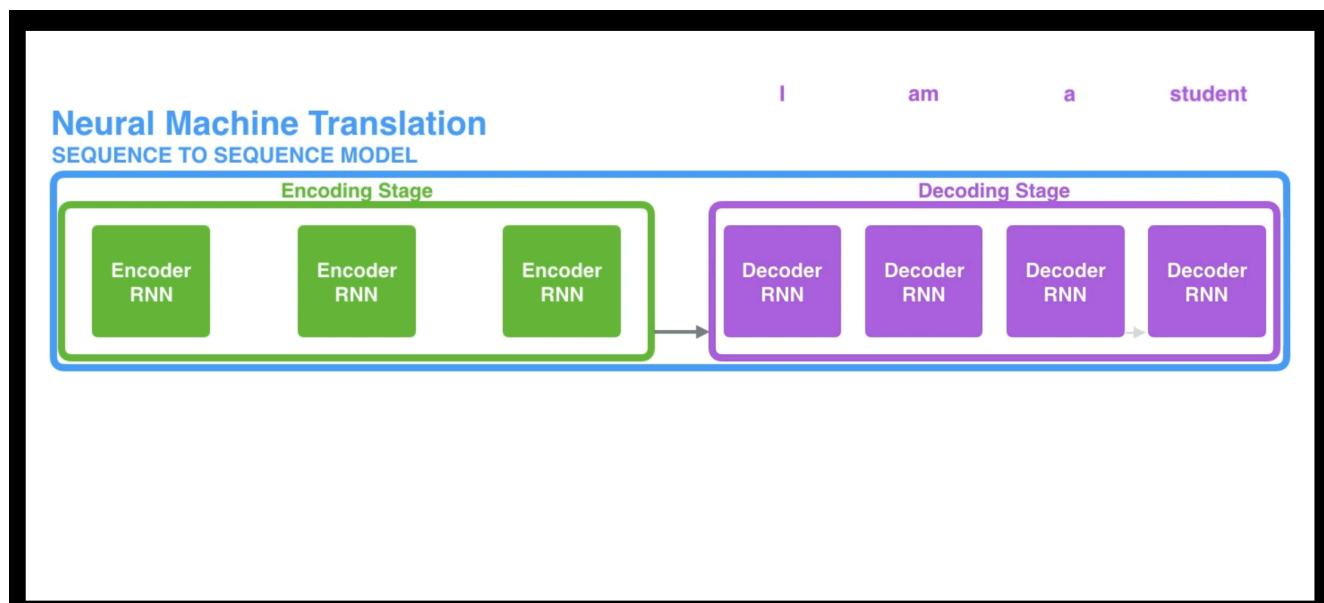
Ключевые проблемы RNN/LSTM:

1. **Проблема бутылочного горлышка (Bottleneck):** Вектор фиксированной длины h_T на последнем шаге должен вместить в себя смысл *всего* прочитанного предложения/временного ряда. При длинных текстах/рядах сеть неизбежно «забывает» начало.
2. **Невозможность распараллеливания:** Вычисления на шаге t зависят от результатов шага $t - 1$. Обучение на длинных последовательностях работает медленно, так как нельзя задействовать всю мощь современных GPU, которые созданы для параллельных матричных вычислений.
3. **Затухание градиентов:** Сложность установления зависимостей между элементами последовательности, которые находятся далеко друг от друга (даже с LSTM эта проблема решается лишь частично).

🔗 Визуализации отсюда

[Jay Alammar – Visualizing machine learning one concept at a time.](#)

[seq2seq6.mp4](#): RNN передает только один hidden state



☰ Example

Суточный паттерн — зависимость шага t от шага $t - 24$ (т.е. 24 часа назад).
LSTM должна пронести ее через 24 шага.
Недельная зависимость — 168 шагов.

| Механизм внимания (Attention Mechanism)

В 2014-2015 годах Бахданау и Луонг для решения проблемы бутылочного горлышка был предложен **механизм внимания**.

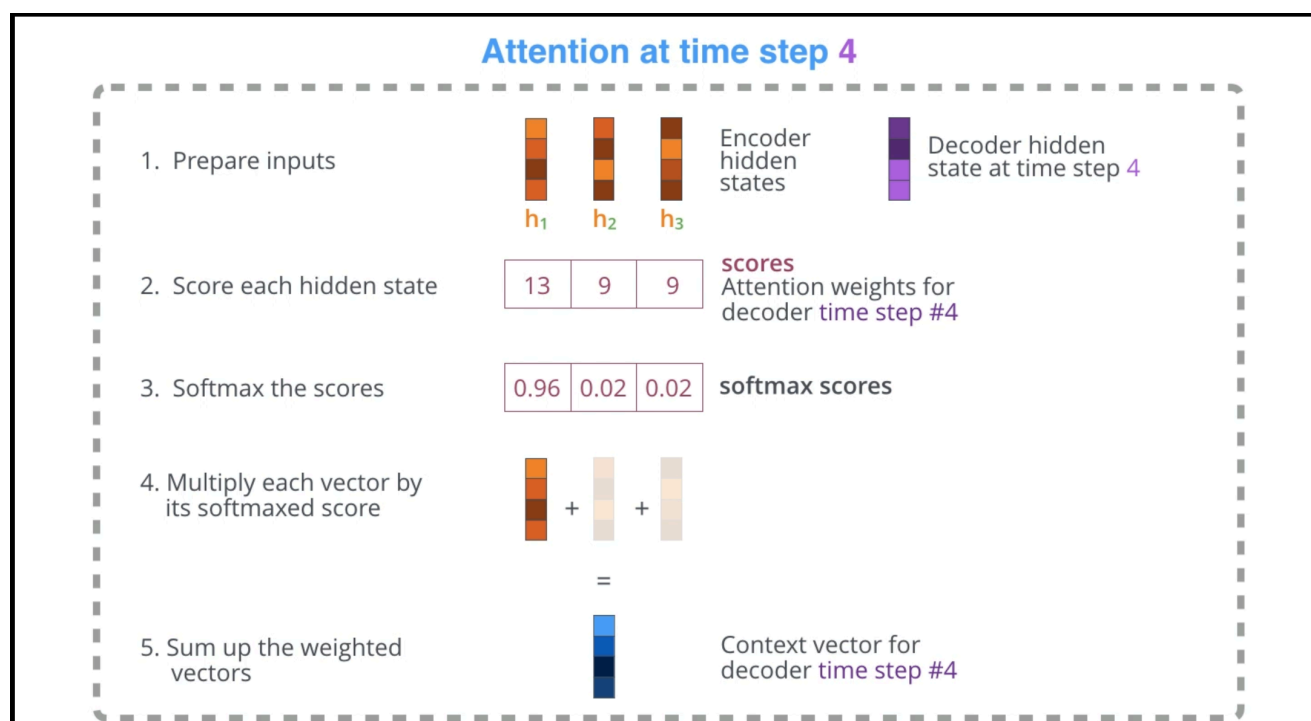
Идея: вместо того, чтобы декодер при генерации смотрел только на один финальный вектор энкодера, давайте дадим ему возможность «смотреть» (*обращать внимание*) на скрытые состояния **всех** элементов исходной последовательности на каждом шаге генерации.

Для прогноза на шаге t вычисляем **вес внимания** $\alpha_{t,s}$ каждого прошлого момента s :

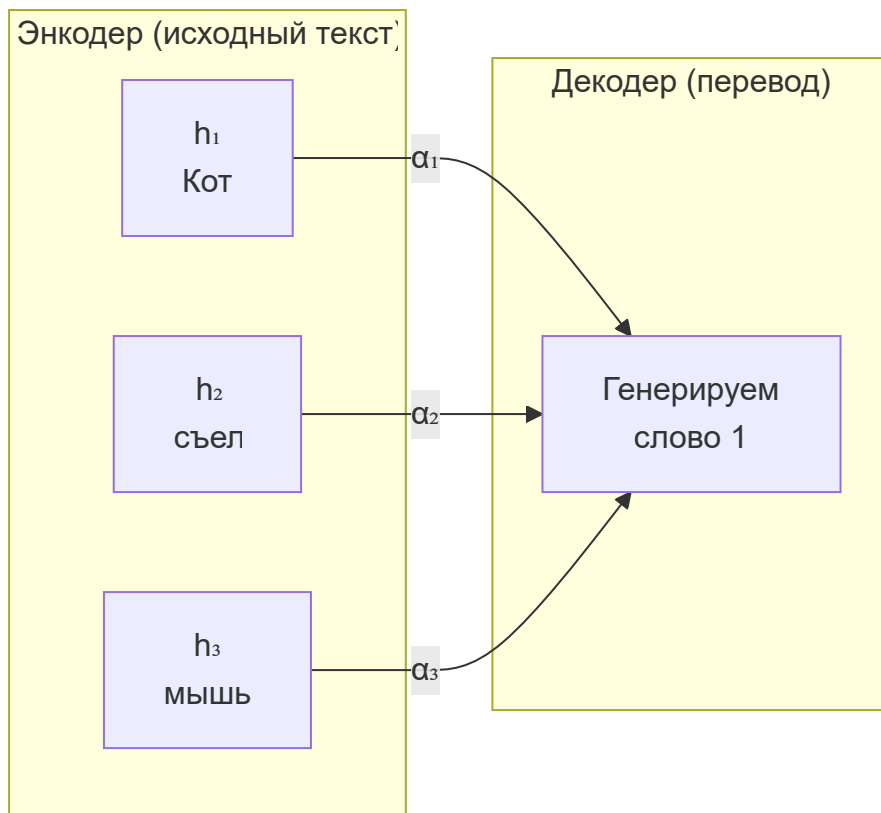
$$e_{t,s} = \text{score}(h_t, h_s), \quad \alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{s'} \exp(e_{t,s'})}$$
$$\text{context}_t = \sum_s \alpha_{t,s} h_s$$

Контекстный вектор context_t — взвешенная сумма всех прошлых состояний.

[Декодер смотрит на все шаги энкодера](#)



Визуализация весов внимания как тепловой карты по времени: [Visualizing A Neural Machine Translation Model](#)



Веса α_i показывают, на какое слово смотрит декодер при генерации каждого выходного токена — это и есть **внимание**.

Формальная реализация механизма внимания в трансформерах: концепция Q, K, V (Query, Key, Value)

В 2017 году исследователи из Google выпустили статью **«Attention Is All You Need»** (Vaswani et al.), где предложили полностью отказаться от рекуррентных сетей и построить архитектуру **исключительно** на механизме внимания — **Transformer**.

В архитектуре Transformer механизм внимания формализован через метафору поиска в базе данных:

1. **Query (Запрос, Q)**: То, что мы ищем (например, слово, для которого сейчас вычисляется контекст).
2. **Key (Ключ, K)**: Теги/описания того, что хранится в базе (все остальные слова в предложении).
3. **Value (Значение, V)**: Сама информация, которую мы извлекаем.

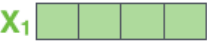
Для каждого токена создаются три вектора (q, k, v) путем умножения исходного эмбединга токена на обучаемые матрицы весов (W^Q, W^K, W^V) .

Input

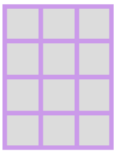
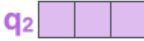
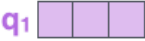
Thinking

Machines

Embedding

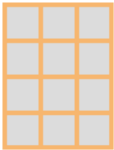
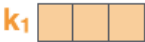


Queries



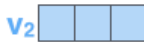
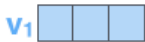
W^Q

Keys



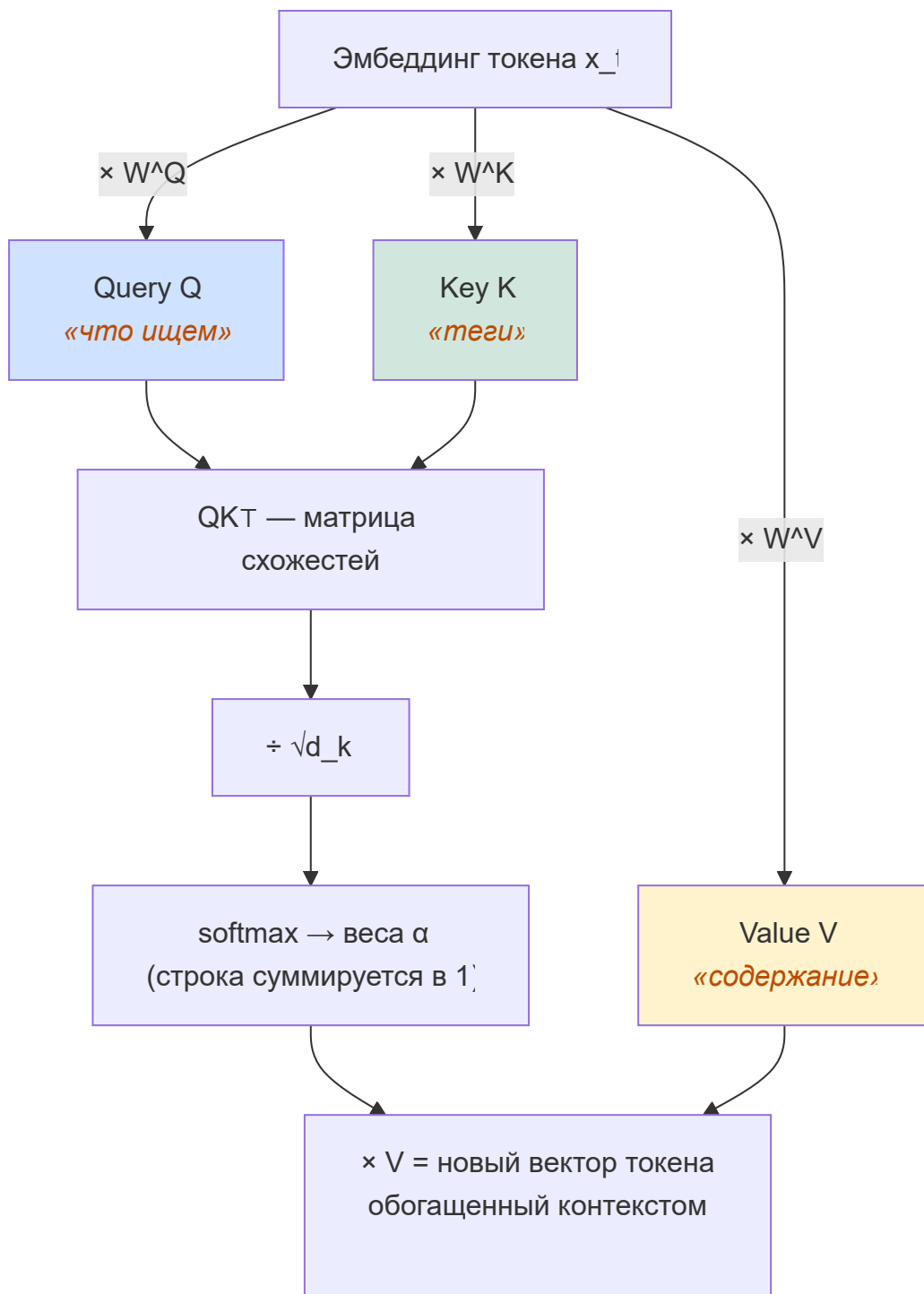
W^K

Values



W^V

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$



🔗 Интерпретация для временных рядов

Строка t матрицы весов внимания показывает, на какие прошлые моменты «смотрит» шаг t при формировании своего представления.

Если ряд имеет суточную периодичность → строка t должна иметь высокий вес у $t - 24$, $t - 48$, $t - 72$; если недельную → у $t - 168$.

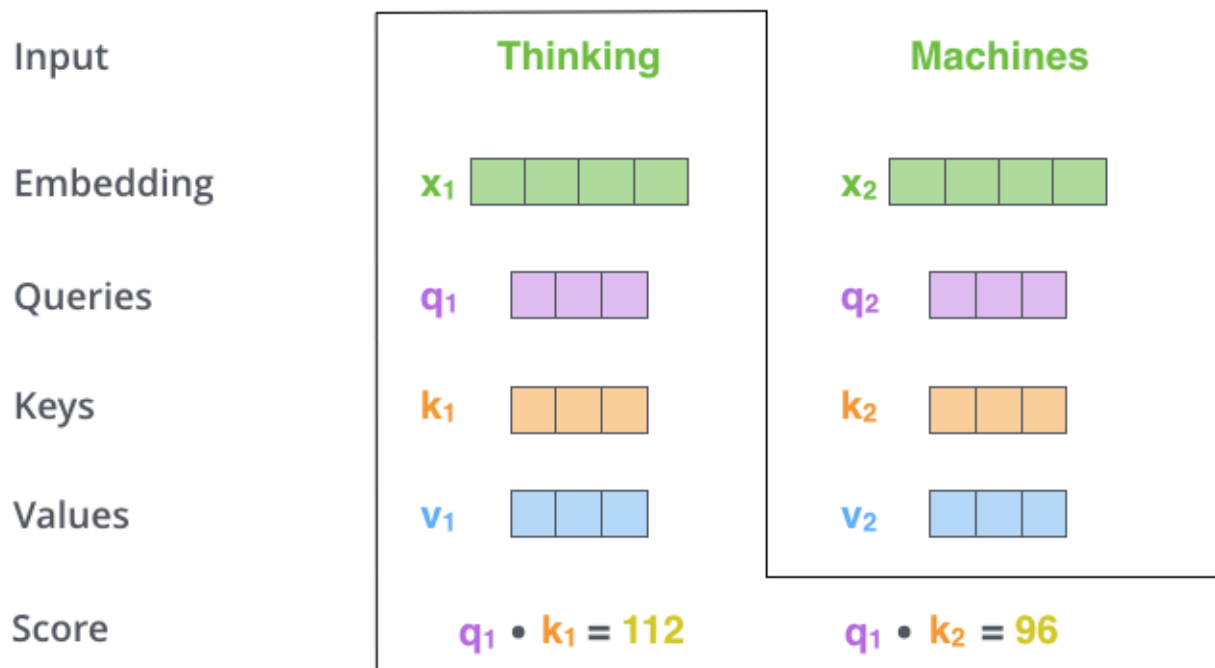
| Self-Attention и Multi-Head Attention в Transformer

| Self-Attention (Самовнимание)

Self-Attention позволяет каждому элементу в последовательности смотреть на **все остальные элементы в этой же последовательности**, чтобы лучше понять свой собственный смысл в данном контексте.

Example

«Он вставил ключ в **замок**» vs «Вдали виднелся старинный **замок**». В первом случае слово «замок» обратит сильное внимание на слова «вставил» и «ключ», а во втором — на «старинный».



Scaled Dot-Product Attention:

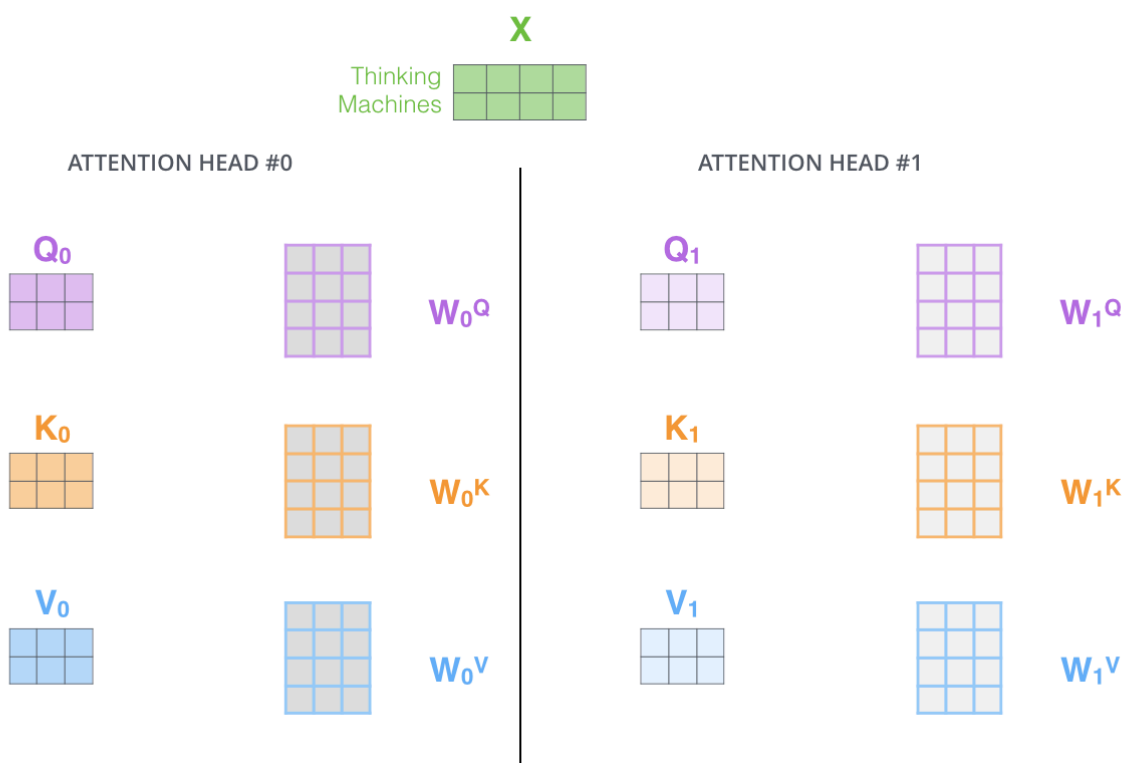
1. Считаем скалярное произведение (QK^T) каждого запроса с каждым ключом — получаем матрицу «сырых» оценок связи между всеми словами.
2. Делим на $\sqrt{d_k}$ (размерность ключа), чтобы предотвратить слишком большие значения, ломающие градиенты softmax.
3. Применяем **softmax** по строкам — получаем **attention weights** (веса внимания от 0 до 1, сумма которых равна 1).
4. Умножаем веса на матрицу значений (V). Итог — новое представление каждого слова, обогащенное контекстом.

$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V = Z$$

Multi-Head Attention (головы внимания)

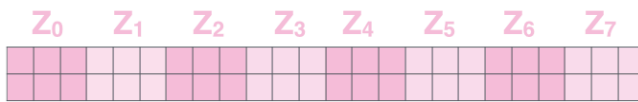
Вместо одного механизма внимания (одной «головы») трансформер использует несколько — механизм **Multi-Head Attention**.

Векторы Q , K и V линейно проецируются h раз в подпространства меньшей размерности, и Self-Attention выполняется параллельно h раз.



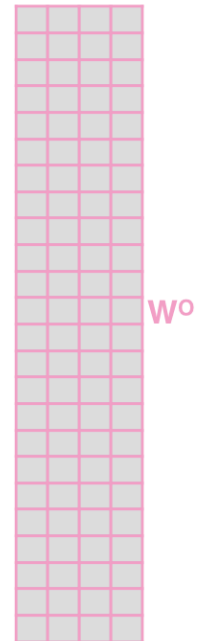
Это позволяет модели одновременно обращать внимание на *разные типы* связей (одна голова следит за грамматикой, другая — за смыслом, третья — за местоимениями). Результаты всех голов конкатенируются.

1) Concatenate all the attention heads

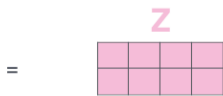


2) Multiply with a weight matrix W^O that was trained jointly with the model

x

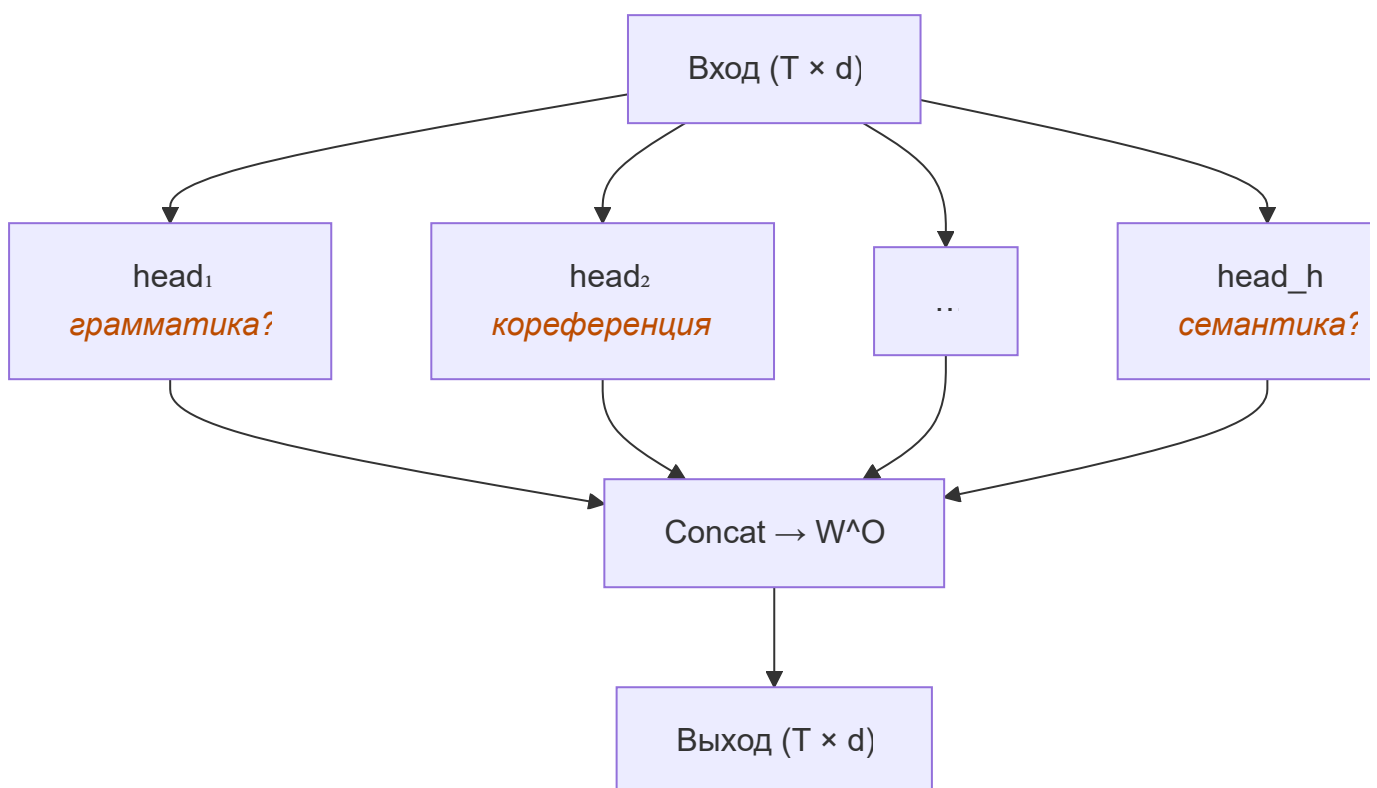


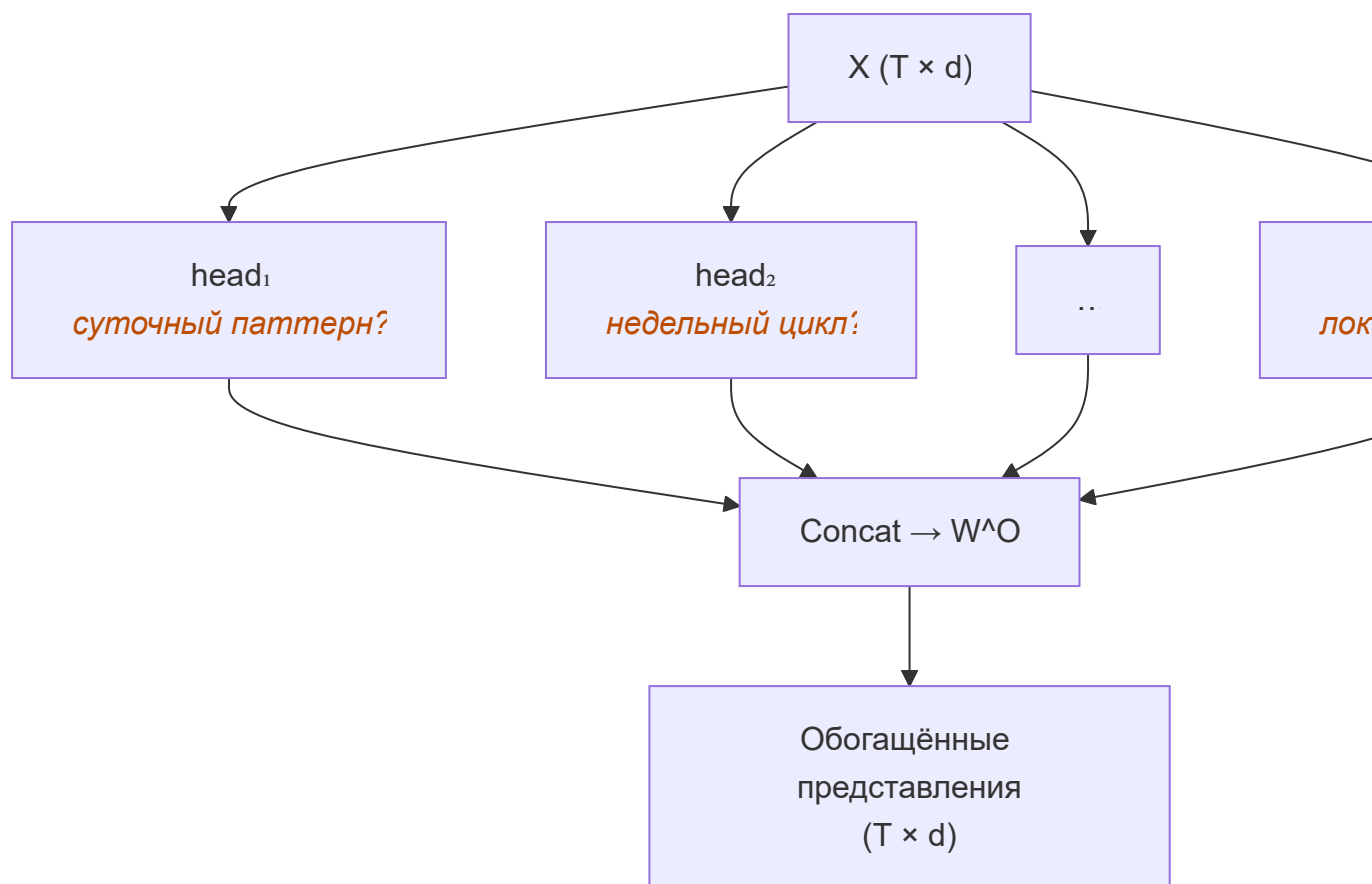
3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



Каждая голова специализируется на своём типе связи.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$





| Архитектура Transformer

Трансформер — архитектура нейронной сети, эффективно решающая задачи «последовательность-в-последовательность» (seq2seq)

Название «трансформер» (преобразователь) соотносится с тем, что основной задачей такой модели является преобразование одной (входной) последовательности в другую (выходную).

Ключевой работой, позволившей развивать данную архитектуру, является статья «Attention Is All You Need» 2017 года [\[1706.03762\] Attention Is All You Need](#)

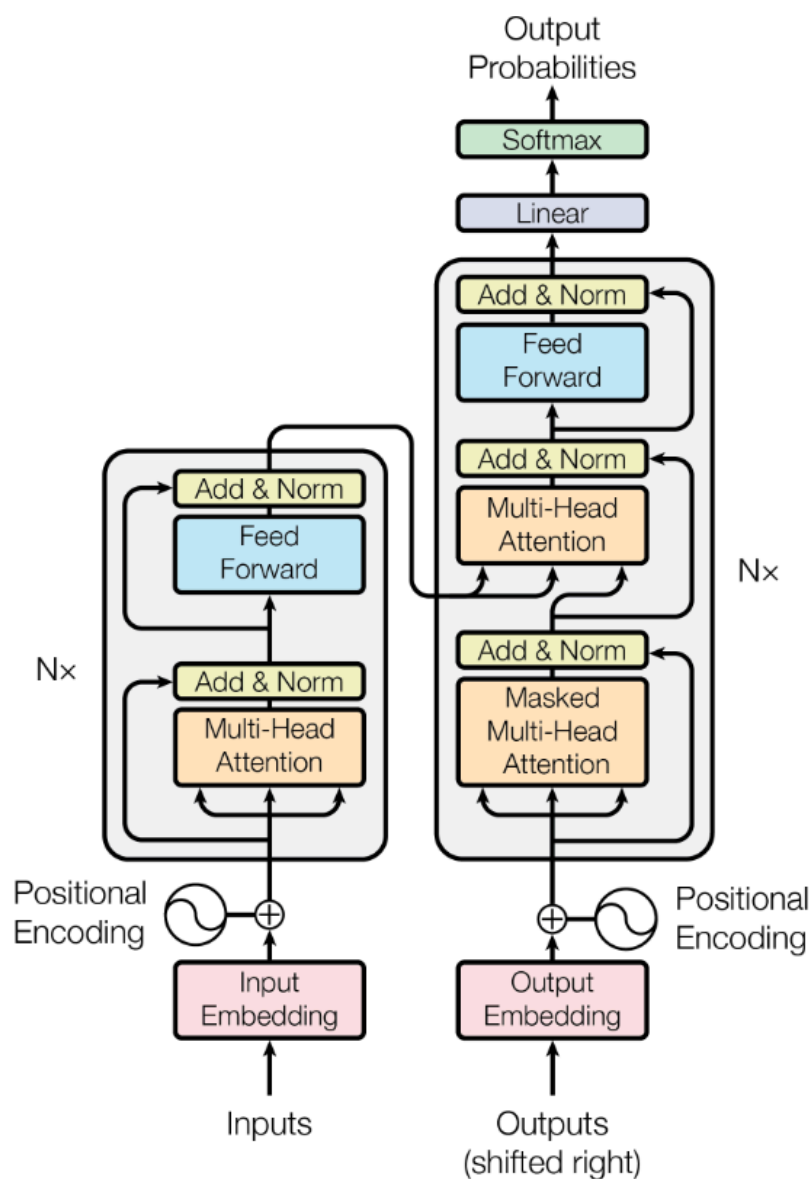


Figure 1: The Transformer - model architecture.

Классический трансформер состоит из **Энкодера** (считывает исходный текст) и **Декодера** (генерирует результат).

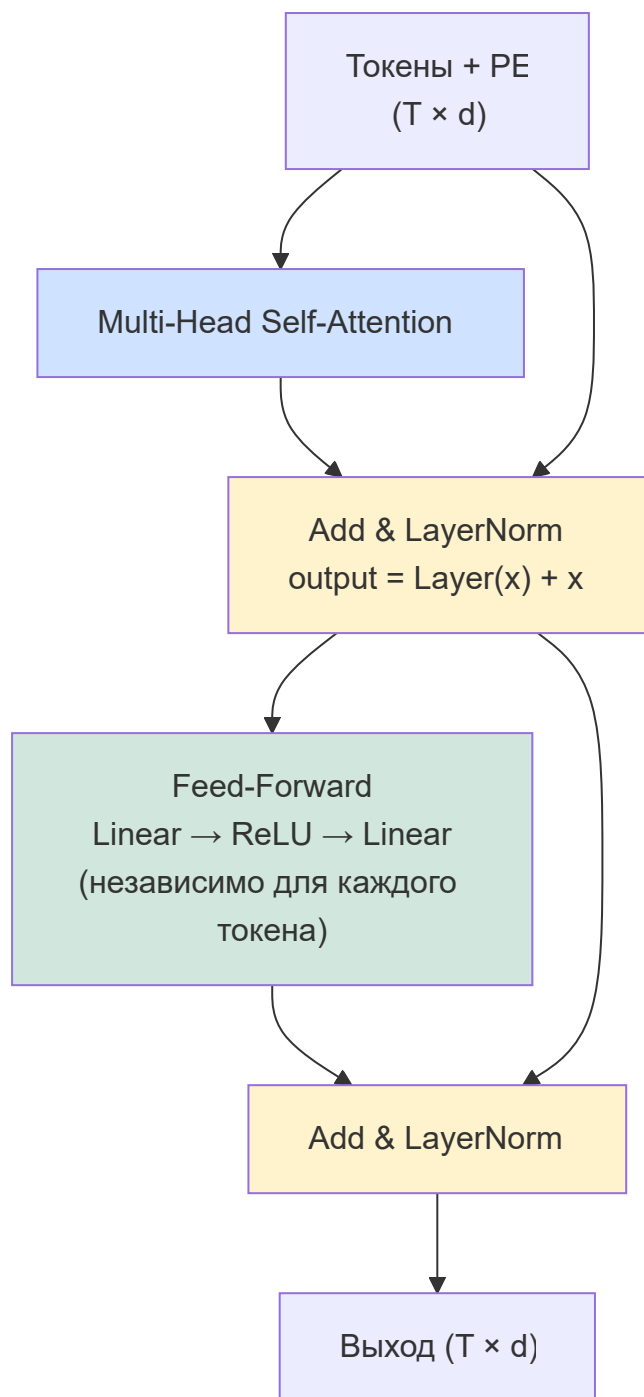
Позиционное кодирование (Positional Encoding)

Трансформер не использует рекуррентные слои и обрабатывает все элементы **одновременно**, поэтому модель изначально не знает, в каком порядке они идут («Кот съел мышь» и «Мышь съела кота» для нее неотличимы).

Чтобы дать модели информацию о порядке, к эмбедингам элементов добавляются специальные векторы позиций, вычисляемые через функции синуса и косинуса.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

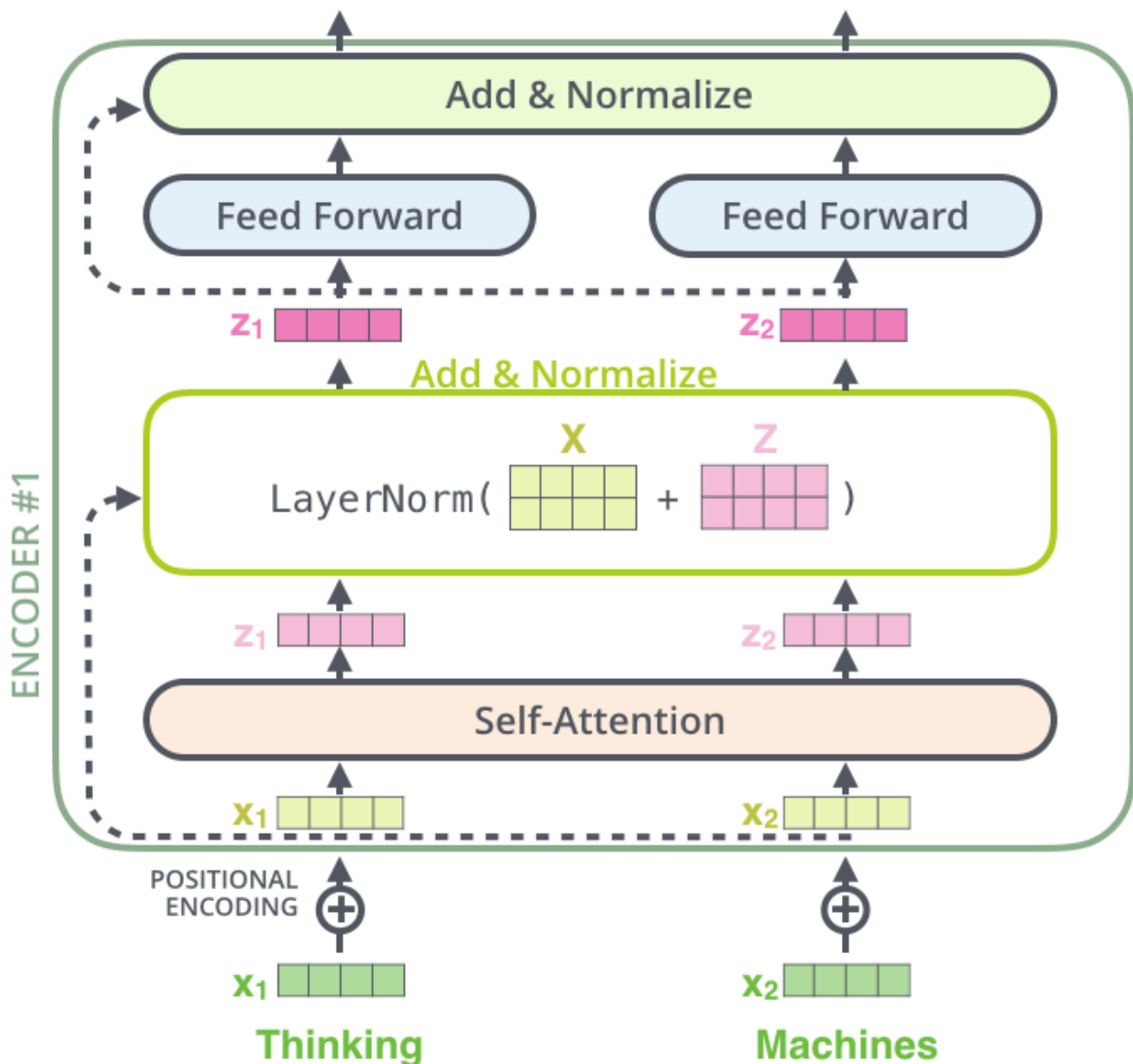
Энкодер



Энкодер состоит из N одинаковых слоев (в оригинале $N = 6$).

Внутри каждого слоя:

1. **Multi-Head Self-Attention** — головы внимания
2. **Feed-Forward Neural Network (FFNN)** — обычная полносвязная сеть (перцептрон), применяемая независимо к каждому токenu.



Между этими подблоками используются:

1. **Residual connections (остаточные связи):** $Output = Layer(x) + x$. Помогают бороться с затуханием градиента в глубоких сетях.
2. **Layer Normalization:** Нормализация активаций для стабилизации обучения.

 Tip

[The Illustrated Transformer](#)

Ветви архитектуры

Оригинальный трансформер был создан для машинного перевода через пару Encoder + Decoder.

Но вскоре выяснилось, что его части можно использовать **по отдельности** для других задач. Так появились три основные ветви развития архитектуры.

| Ветвь 1: Encoder-only (ветвь понимания) — BERT, RoBERTa

Используют **только энкодер** трансформера.

Особенность: двунаправленное внимание (Bidirectional Attention). Токен на позиции i может «смотреть» как влево, так и вправо.

Идеально для задач понимания текста — классификации, анализа тональности, извлечения именованных сущностей (NER), ответа на вопросы (когда ответ надо найти в тексте).

Минус: не умеют генерировать связный текст авторегрессионно.

| Ветвь 2: Decoder-only (ветвь генерации) — GPT, LLaMA, Mistral

Используют **только декодер**.

Особенность: однонаправленное внимание. Токен на позиции i может смотреть **только влево** (на предыдущие токены). Взгляд в будущее «замаскирован» (Causal/Masked Attention).

Идеально для задач генерации текста — модель предсказывает следующее слово на основе предыдущих. Большинство современных LLM (ChatGPT, Claude, Deepseek и т.д.) построены по этой схеме.

| Encoder-Decoder (универсальная ветвь) — T5, BART

Сохраняют оригинальную структуру.

Особенность (подход T5 - Text-to-Text Transfer Transformer): любая задача формулируется как «текст на входе → текст на выходе». Например, на входе `translate English to German: That is good`, на выходе `Das ist gut`.

Идеально для задач машинного перевода, реферирования (пересказ текста).

| Трансформеры специально для временных рядов

Стандартный трансформер не учитывает специфику TS: нет встроенной сезонности, токены — скалярные значения, а не слова.

Адаптации:

Архитектура	Идея	Ссылка
Informer (2021)	ProbSparse Attention: $O(T \log T)$ вместо $O(T^2)$	arxiv 2012.07436
PatchTST (2023)	Разбить ряд на патчи (как ViT — изображение)	arxiv 2211.14730
TFT (2020)	Явный учет сезонности + интерпретируемость	arxiv 1912.09363

Лабораторная работа №6. Прогнозирование временных рядов

Цель: многошаговый прогноз. По окну из $L = 96$ значений предсказать следующие $H = 24$:

$$\hat{x}_{t+1}, \dots, \hat{x}_{t+24} = f(x_{t-95}, \dots, x_t)$$

Датасет: **ETTh1** (Electricity Transformer Temperature, почасовые данные 2016–2018)

```
import pandas as pd

url = "https://raw.githubusercontent.com/zhouhaoyi/ETDataset/main/ETT-small/ETTh1.csv"
df = pd.read_csv(url, parse_dates=["date"], index_col="date")
series = df["OT"]  # Oil Temperature – одномерный прогноз, ~17 420 точек
```

Разбивка: 70% train → 10% val → 20% test.

`MinMaxScaler` обучается **только на train** — тот же принцип data leakage, что в предыдущих лабораторных.

1. Наивное предсказание

```
import numpy as np

def naive(window, H):
    return np.full(H, window[-1])

def roll_mean(window, H):
    return np.full(H, window.mean())
```

Вычислить MAE и RMSE на тестовом множестве:

$$\text{MAE} = \frac{1}{NH} \sum_i \sum_{h=1}^H |\hat{x}_h^{(i)} - x_h^{(i)}|, \quad \text{RMSE} = \sqrt{\frac{1}{NH} \sum_i \sum_{h=1}^H (\hat{x}_h^{(i)} - x_h^{(i)})^2}$$

2. LSTM

```
import torch
from torch.utils.data import Dataset

class TSDataset(Dataset):
    def __init__(self, data, L=96, H=24):
        self.data = torch.tensor(data, dtype=torch.float32)
        self.L, self.H = L, H
```



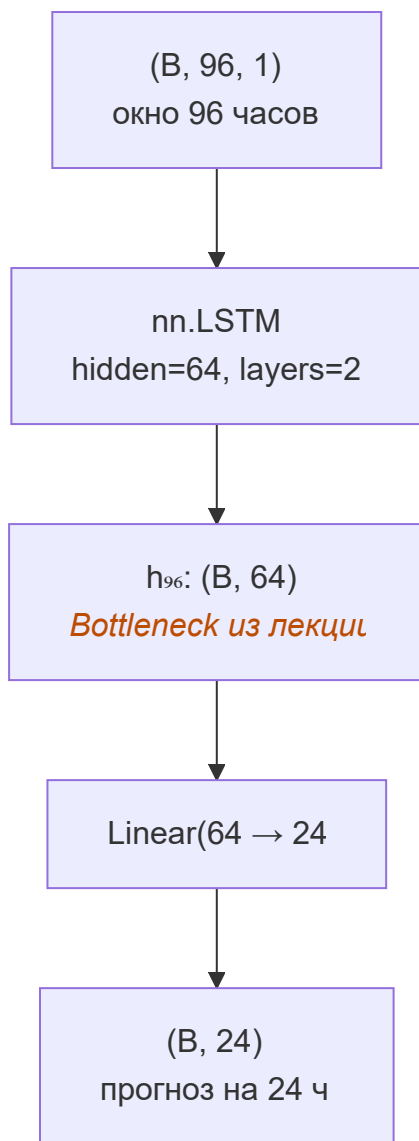
```
def __len__(self):
    return len(self.data) - self.L - self.H + 1

def __getitem__(self, idx):
    x = self.data[idx : idx + self.L].unsqueeze(-1)      # (L, 1)
    y = self.data[idx + self.L : idx + self.L + self.H]  # (H,)
    return x, y
```

```
import torch.nn as nn

class LSTMForecaster(nn.Module):
    def __init__(self, hidden=64, layers=2, H=24, dropout=0.2):
        super().__init__()
        self.lstm = nn.LSTM(1, hidden, layers, batch_first=True,
                             dropout=dropout)
        self.head = nn.Linear(hidden, H)

    def forward(self, x):
        out, _ = self.lstm(x)
        return self.head(out[:, -1])
```



Обучить 30 эпох, `AdamW(lr=1e-3)`, `MSELoss`.

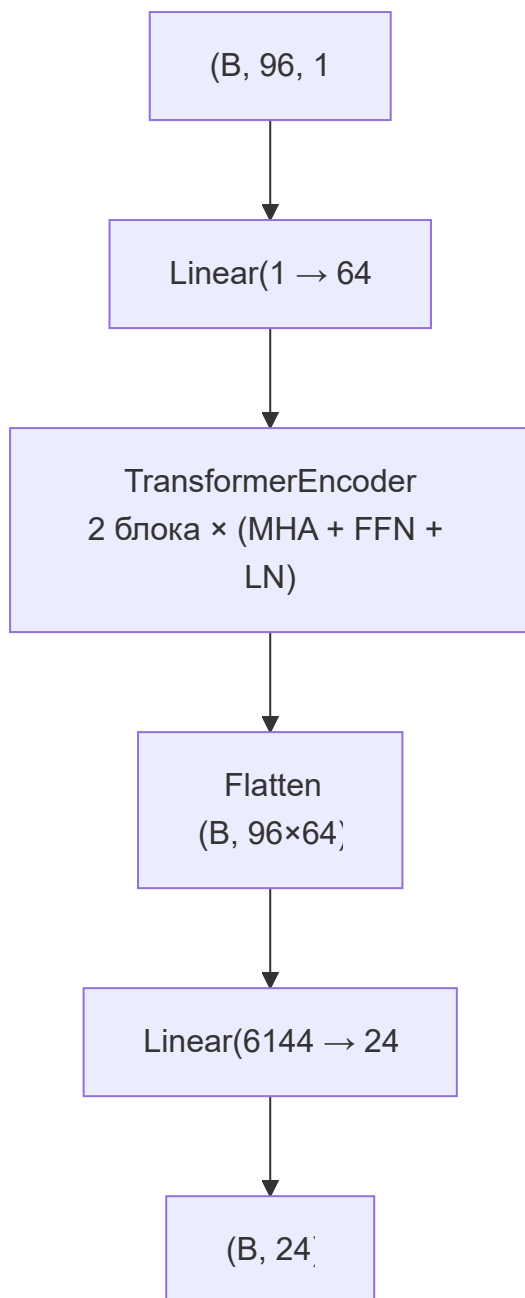
Оставляем визуализации train/val loss по эпохам, MAE и RMSE на test, 5 случайных окон из test: факт vs прогноз (subplot)

3. Transformer

```
class TSTransformer(nn.Module):
    def __init__(self, d_model=64, nhead=4, layers=2, L=96, H=24):
        super().__init__()
        self.proj = nn.Linear(1, d_model)
        enc = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead, dim_feedforward=256,
            dropout=0.1, batch_first=True
        )
        self.encoder = nn.TransformerEncoder(enc, num_layers=layers)
        self.head = nn.Linear(d_model * L, H)

    def forward(self, x):
        x = self.proj(x)
```

```
x = self.encoder(x)
return self.head(x.flatten(1))
```



⚠ Warning

`nn.TransformerEncoderLayer` не добавляет positional encoding автоматически. Для этой задачи можно добавить `nn.Embedding(L, d_model)` как обучаемое позиционное кодирование.

Ссылки

1. [ETTh1 датасет](#) (~1.4 MB)
2. [Informer paper — датасет ETT](#)
3. [PyTorch LSTM](#)
4. [PyTorch TransformerEncoder](#)

| Большие языковые модели (LLM) и RAG-системы

На прошлой лекции мы говорили о BERT (**Encoder-only**), который учится заполнять пропуски в середине текста.

Сегодня речь пойдет о **Decoder-only** архитектурах (семейство GPT: от оригинального GPT-1 до ChatGPT, Claude, Mistral).

Их главная задача в процессе обучения — **каузальное языковое моделирование (Causal Language Modeling)**.

==**Суть**: модель учится предсказывать **следующий токен** на основе **всех предыдущих**. ==

Разумеется, как в случае с Encoder-подходом, используются маски: взгляд в «будущее» замаскирован (Masked Self-Attention), поэтому модель не может подсмотреть следующее слово.

Когда модель обучена, она выполняет процедуру **авторегрессионной генерации**:

1. Пользователь подает на вход модель текст: **Мама мыла**
2. Модель выдает распределение вероятностей для следующего слова. Наиболее вероятным оказывается **раму**
3. Слово **раму** добавляется к входному тексту: **Мама мыла раму**
4. Шаги 2 и 3 **повторяются** до тех пор, пока модель не сгенерирует **специальный токен окончания последовательности** (например, **<EOS>** — End of Sequence)

Стратегии семплирования

Если всегда брать слово с максимальной вероятностью (прибегать к жадному поиску (Greedy search)), то текст получится однообразным и зацикленным.

Для борьбы с этим используют параметры, меняющие принципы выбора следующего токена:

1. **Температура (Temperature)**: параметр, сглаживающий распределение. При $T \rightarrow 0$ модель становится детерминированной (выбирает самое вероятное), при $T > 1$ модель начинает рисковать и выдавать редкие слова («креативить»).
2. **Top-K**: модель рассматривает только K наиболее вероятных слов на каждом шаге.
3. **Top-P (Nucleus sampling)**: модель рассматривает минимальный набор слов, сумма вероятностей которых превышает P (например, 0.9).
4. **Min-P**: модель рассматривает только токены, вероятность которых превышает P (например, 0.05).

| Этапы обучения современных LLM

1. Pre-training (обучение Base-модели)

Модель обучается на огромных корпусах текстов (терабайты — «весь интернет») и учится предсказывать следующий токен.

В результате модель знает язык и факты, но **не умеет отвечать на вопросы**.

Example

Если спросить базовую модель: «Как сварить пельмени?», она может сгенерировать продолжение: «...спрашивали пользователи на форуме домохозяек в 2012 году».

Базовые модели просто продолжают текст.

2. Supervised Fine-Tuning (SFT / Instruct-tuning)

Модель дообучают на тысячах высококачественных примеров диалогов:

(Пользователь: вопрос) → (Ассистент: качественный и правильный ответ на вопрос пользователя) .

В результате модель понимает формат диалога и начинает выполнять инструкции пользователя.

При публикации такие модели часто имеют суффикс **-Instruct** или **-Chat** .

3. RLHF (Reinforcement Learning from Human Feedback — обучение с подкреплением на основе обратной связи от человека)

Это этап «выравнивания» (Alignment) модели с определенными человеческими ценностями

1. Люди оценивают разные варианты ответов модели на один промпт (что лучше, что хуже)
2. На этих оценках обучается вспомогательная Reward Model (модель награды)
3. Основная модель дообучается методом обучения с подкреплением (например, алгоритмом PPO), стараясь максимизировать награду от Reward-модели

В результате получаются современные модели вроде ChatGPT или Claude, которые стремятся быть «безопасными», «полезными» и «не токсичными»

Important

Такой подход к наградам очень часто приводит к избыточной сикофантия (sycophancy) — «поддакиванию» пользователю.

Модель с типичным элайментом склонна превозносить пользователя, хвалить его,

соглашаться с ним и поддерживать его точку зрения.

Это может вести к превращению взаимодействия в «эхо-камеру», когда ввод пользователя де-факто возвращается ему, просто усиленный согласием и похвалой модели

| Проблемы больших языковых моделей

1. **Галлюцинации**: модель не «знает» правду, она лишь предсказывает статистически вероятный следующий токен. Если в обученной модели нет какого-либо факта, сгенерирует грамматически безупречную, но фактически неверную информацию. Это усугубляется тем, что вся процедура тренировки моделей направлена на продолжение диалога и выдачу ответа пользователю.
2. **Замороженные знания**: доступная модели информация зафиксирована на моменте завершения ее обучения. Модель, обученная в 2025 году, не знает, что произошло в 2026.
3. **Ограниченное контекстное окно**: вычислительная сложность Attention растет квадратично от длины текста, поэтому контексты окна моделей имеют определенный предел (например, 8k, 32k или 128k токенов — но сейчас есть передовые модели с контекстами в 1 млн. токенов)

| Архитектура RAG (Retrieval-Augmented Generation)

Для решения проблем галлюцинаций и актуальности данных в 2020 году предложена архитектура **RAG (Retrieval-Augmented Generation — генерация, дополненная поиском)**.

Суть: модель получает доступ к некоторой внешней базе данных. Перед тем, как генерировать ответ, модель обращается к этой базе через систему поиска и извлечения информации (retrieval), а затем, когда информация найдена или явно указано, что найти ее не удалось, результат этого поиска добавится (augment) в промпт при генерации ответа.

| Этапы RAG

| 1. Индексация

1. Берем базу знаний (множество документов)
2. Разбиваем тексты на небольшие куски (чанки — chunks, например, по 500 слов)
3. Прогоняем каждый чанк через модель-энкодер (эмбединг-модель), получая плотный вектор
4. Сохраняем тексты и их векторы в векторную базу данных (ChromaDB, FAISS, Milvus, PostgreSQL + pg-vector, sqlite с векторными расширениями (sqlite_vec) и т.п.)

| 2. Поиск (Retrieval)

1. Пользователь задает вопрос: «Какие правила оформления отпуска?»
2. Вопрос пропускается через эмбединг-модель, получаем вектор вопроса
3. Векторная БД ищет (например, через косинусное сходство) Топ- K чанков, векторы которых максимально близки к вектору вопроса

3. Генерация (Generation)

1. Найденные тексты объединяются с оригинальным вопросом в единый промпт
2. Этот промпт отправляется в LLM
3. Она генерирует точный ответ со ссылкой на источники

Note

Ранее требовались специальные модели, которые понимали структуру RAG-ответа вроде:

Ты ИИ-ассистент. Используй только следующий контекст для ответа. Контекст: [вставка Топ- K найденных кусков]. Вопрос: [вопрос пользователя]

Современные модели обычные поддерживают RAG без дополнительных усилий